

Stack technique IMSOP

Ce que chaque brique fait, pourquoi elle est là, et ce que tu peux reprendre pour tes prochains projets. Les versions citées sont celles réellement installées dans le projet.

1. Les trois que tu voulais comprendre

Prisma, Axios et Zustand résolvent trois problèmes différents. Une fois que tu vois le problème, l'outil devient évident.

Axios — parler au serveur

Son rôle : Envoyer des requêtes HTTP depuis le navigateur vers ton API.

Le problème qu'elle résout : Ton interface React tourne dans le navigateur. Ta base de données, elle, est sur le serveur. Les deux ne peuvent pas se parler directement : il faut envoyer une requête HTTP et attendre la réponse. Axios fait ce trajet, convertit le JSON automatiquement, et surtout permet de configurer une fois pour toutes l'adresse du serveur et le jeton d'authentification, au lieu de les répéter à chaque appel.

Dans IMSOP : Le fichier `src/lib/api.js` configure Axios une seule fois. Ensuite, partout dans le code, `api.post('/dossiers', {...})` suffit : le jeton JWT du médecin connecté est attaché automatiquement. C'est cette ligne qui envoie la demande de deuxième avis au backend.

L'image qui aide

C'est le service postal entre le navigateur et le serveur. Tu écris la lettre ; Axios connaît l'adresse, colle le timbre, et te rapporte la réponse.

Zustand — la mémoire partagée de l'interface

Son rôle : Stocker les informations dont plusieurs écrans ont besoin en même temps.

Le problème qu'elle résout : En React, chaque composant a sa propre mémoire, isolée. Mais certaines informations concernent tout le monde : qui est connecté, son nom, son rôle. Sans outil, il faudrait faire descendre cette information de composant en composant, à la main, sur toute la profondeur de l'arbre. C'est long et fragile. Zustand la met dans un endroit unique que n'importe quel composant peut lire directement.

Dans IMSOP : Le magasin `useAuthStore` contient l'utilisateur connecté. C'est grâce à lui que `ConsentementGate` connaît le nom du titulaire du compte (`s.user?.fullName`) pour vérifier que la signature correspond bien, sans que personne n'ait eu à le lui transmettre.

L'image qui aide

C'est le tableau blanc du bureau. Au lieu de faire passer l'information de bureau en bureau, on l'écrit au mur une fois : tous ceux qui en ont besoin la lisent.

Prisma — le pont vers la base de données

Son rôle : Traduire entre ton code JavaScript et ta base PostgreSQL, et garder les deux synchronisés.

Le problème qu'elle résout : Ta base parle SQL. Ton code parle JavaScript. Écrire du SQL à la main est possible, mais fastidieux et risqué : une faute de frappe dans un nom de colonne ne se voit qu'à l'exécution, parfois en production. Prisma résout trois choses à la fois : il décrit toutes tes tables au même endroit, il génère le code d'accès, et il fabrique les migrations qui font évoluer la vraie base.

Dans IMSOP : Quand le champ allergies a été ajouté, une seule ligne a changé dans `schema.prisma` : `allergies String?`. Prisma a généré la migration `20260907103000_allergies_dossier`, qui contient le SQL `ALTER TABLE "Dossier" ADD COLUMN "allergies" TEXT`. Ensuite `prisma.dossier.create({ data: { allergies } })` a fonctionné sans une seule ligne de SQL écrite à la main.

L'image qui aide

C'est à la fois le traducteur et le chef de chantier. Le traducteur convertit tes phrases JavaScript en SQL. Le chef de chantier vérifie que le bâtiment (la base) correspond toujours aux plans (`schema.prisma`), et garde l'historique de chaque modification.

Les trois ensemble, sur un seul geste

Un médecin clique sur « Envoyer à la coordination » :

1	Zustand	fournit l'identité du médecin connecté, pour vérifier sa signature.
2	Axios	envoie le formulaire au serveur, avec le jeton d'authentification.
3	Zod	côté serveur, vérifie que chaque champ est valide avant d'aller plus loin.
4	Prisma	écrit le dossier dans PostgreSQL et renvoie l'objet créé.

2. L'inventaire complet

Frontend — ce que voit l'utilisateur

Technologie	À quoi ça sert
React 18	La bibliothèque qui construit l'interface. Tout le dossier <code>src/</code> est en React.
Vite 5	Serveur de développement (port 5173) et compilateur pour la production. C'est lui qui tourne avec <code>npm run dev</code> et <code>npm run build</code> .
React Router 6	La navigation entre les pages. C'est ce qui a été modifié pour que le bouton « DEMANDER UN AVIS » mène vers l'inscription.
Tailwind CSS 3	Les styles écrits directement en classes dans le JSX, sans fichier CSS séparé par composant.
PostCSS, autoprefixer	La tuyauterie qui exécute Tailwind et ajoute les préfixes navigateurs. Tu n'y touches jamais.

Composants et apparence

Technologie	À quoi ça sert
Radix UI	Des composants sans style mais accessibles : fenêtres modales, menus, infobulles. Tu poses ton style Tailwind par-dessus.
lucide-react	La bibliothèque d'icônes.
framer-motion	Les animations.
clsx, tailwind-merge, class-variance-authority	Trio utilitaire pour combiner des classes Tailwind sans qu'elles se contredisent. Utile dès que tu fais un composant à variantes, par exemple un bouton primaire et un bouton secondaire.

État et communication côté client

Technologie	À quoi ça sert
Zustand 4	L'état global : l'utilisateur connecté vit là.
Axios 1	Les appels HTTP vers l'API.
Socket.IO client 4	Le temps réel : messagerie, notifications, signalisation des appels vidéo.
i18next, react-i18next	Le bilingue français et anglais. Tous les textes passent par <code>t('...')</code> et vivent dans <code>src/i18n/locales/</code> .

Backend — le serveur

Technologie	À quoi ça sert
Node.js 22	Le moteur qui exécute le JavaScript côté serveur.
Express 4	Le cadre qui définit les routes de l'API.
PostgreSQL	La base de données.
Prisma 5	L'ORM : décrit les tables, génère le code d'accès et les migrations.

Technologie	À quoi ça sert
Zod 3	Valide tout ce qui entre dans l'API. Un champ absent ou trop long est rejeté avant d'atteindre ta logique métier.

Sécurité

Technologie	À quoi ça sert
bcrypt	Hache les mots de passe. Aucun mot de passe n'est jamais stocké en clair.
jsonwebtoken (JWT)	Les jetons de session qui prouvent qu'un utilisateur est bien connecté.
helmet	Pose automatiquement les en-têtes HTTP de sécurité.
express-rate-limit	Limite le nombre de tentatives, contre les attaques par force brute. C'est le point réparé lors du pentest.

Fichiers, documents et communications

Technologie	À quoi ça sert
multer	Réception des fichiers envoyés, par exemple les documents médicaux.
AWS SDK S3	Stockage des fichiers sur du stockage objet plutôt que sur le disque du serveur.
pdfkit	Génération des PDF, notamment les formulaires de consentement.
nodemailer	Envoi des e-mails, dont les codes de vérification à deux facteurs.
Socket.IO 4	Le pendant serveur du temps réel.
node-cron	Tâches planifiées : relances, alertes de délai de réponse.

Tests

Technologie	À quoi ça sert
node --test	Le lanceur de tests intégré à Node, sans aucune dépendance à installer. C'est lui qui exécute backend/tests/.
supertest	Simule des requêtes HTTP sur l'API sans avoir à lancer un vrai serveur.
Vitest, jsdom	Tests des composants React.
Playwright 1.63	Pilote un vrai navigateur pour tester un parcours complet. C'est l'outil utilisé pour vérifier le formulaire de demande de bout en bout.

3. Ce que tu reprends pour tes prochains projets

Tu n'as pas besoin de tout cela à chaque fois. Le choix dépend de ce que ton projet doit faire.

Quand	Ce qu'il te faut	Pourquoi
Toujours, dès le départ	Node, Vite, React, React Router, Tailwind, Axios.	Le socle minimal de n'importe quelle application web moderne.
Dès qu'il y a des comptes	Express, PostgreSQL, Prisma, Zod, bcrypt, JWT.	Authentification, données persistantes, validation des entrées.
Si plusieurs écrans partagent des données	Zustand.	Inutile sur un site simple. Indispensable dès qu'une session utilisateur circule partout.
Si du temps réel	Socket.IO, côté client et côté serveur.	Messagerie, notifications instantanées, appels.
Si bilingue ou multilingue	i18next et react-i18next.	À mettre en place tôt : rajouter le multilingue après coup coûte très cher.
Si envoi de fichiers	multer, puis un stockage objet de type S3.	Ne stocke jamais durablement les fichiers sur le disque du serveur applicatif.

Ce qui est propre au médical

Trois choses de ce projet ne se transposent pas telles quelles ailleurs, parce qu'elles existent à cause de la nature des données de santé : **pdfkit** pour produire les formulaires de consentement signés, la **vérification à deux facteurs par e-mail** au moment de la signature, et la **rigueur de la validation Zod** sur chaque entrée. Sur un projet ordinaire, tu peux être plus souple. Sur des données médicales, non.

Un conseil de méthode

N'installe jamais une bibliothèque « au cas où ». Chaque dépendance est du code que tu ne contrôles pas, qu'il faudra mettre à jour, et qui peut contenir une faille. Pars du problème concret que tu as, et ajoute l'outil seulement quand le faire à la main devient vraiment pénible.